?# Manual Completo - Desperdicio Zero

Versao do manual: 1.0
Data: 2026-03-08
Base de codigo: commit 36c4cef
Versao estavel de referencia: tag versao-01 (commit 68e51ce)

Como usar este manual

Este documento foi escrito para ser didatico e servir dois publicos:

1. Voce, dono do projeto, para aprender arquitetura, operacao e manutencao.
2. Um proximo desenvolvedor, para assumir o sistema sem depender de contexto verbal.

Ordem recomendada de estudo:

1. Entenda visao e arquitetura.
2. Entenda setup local no Windows.
3. Entenda Firebase (auth, aprovacao e regras).
4. Entenda deploy no GitHub Pages.
5. Estude o mapa de arquivos e funcoes-chave.
6. Consulte os apendices de codigo quando quiser profundidade tecnica.

Visao geral do produto

Desperdicio Zero e uma plataforma web para reduzir perdas alimentares com foco em operacao de cozinhas comerciais.

Funcionalidades implementadas:

- area publica com demonstracao;
- cadastro/login;
- painel do cliente;
- painel admin;
- gestao de estoque;
- lista de compras;
- sugestoes de receitas;
- desafios;
- aprovacao de novos clientes por admin;
- verificacao de e-mail;
- recuperacao de senha;
- exportacao de backup JSON.

Secao consolidada: README.md

Desperdicio Zero

MVP web da plataforma **Desperdicio Zero**, com area publica, autenticacao, area do cliente, area administrativa e modulo de Cozinha Inteligente.

Stack

- React + Vite
- TailwindCSS
- Lucide Icons
- Framer Motion
- Firebase Authentication + Firestore (modo online)
- LocalStorage (modo local de fallback)

Versao estavel

- Nome: **Versao 01**
- Tag Git: ‘versao-01‘
- Commit base: ‘68e51ce‘

Estrutura

```
src/
  components/
    admin/
    common/
    layout/
    routing/
    sections/
  pages/
    public/
    auth/
    client/
    admin/
  services/
  hooks/
  utils/
  data/
```

Pre-requisitos

- Node.js 20+
- npm 10+

Rodar localmente

```
npm.cmd install
npm.cmd run dev
```

Build de producao

```
npm.cmd run build
npm.cmd run preview
```

Deploy com GitHub Pages

O workflow em ‘.github/workflows/deploy-gh-pages.yml‘ publica ‘dist‘ no branch ‘gh-

pages‘ a cada push no branch ‘main‘.

Modo de backend

Variavel ‘VITE_BACKEND_PROVIDER‘:

- ‘local‘ (MVP local)
- ‘firebase‘ (auth e banco real)
- ‘supabase‘ (reservado para futuro)

Documentacao completa

- Indice: [‘docs/README.md‘](./docs/README.md)
- Versao de referencia: [‘docs/VERSAO_01.md‘](./docs/VERSAO_01.md)
- Arquitetura: [‘docs/ARQUITETURA_E_FLUXOS.md‘](./docs/ARQUITETURA_E_FLUXOS.md)
- Setup Windows: [‘docs/AMBIENTE_WINDOWS.md‘](./docs/AMBIENTE_WINDOWS.md)
- Firebase: [‘docs/FIREBASE_OPERACAO.md‘](./docs/FIREBASE_OPERACAO.md)
- Deploy: [‘docs/DEPLOY_GITHUB_PAGES.md‘](./docs/DEPLOY_GITHUB_PAGES.md)
- Manutencao/Rollback: [‘docs/MANUTENCAO_E_ROLLBACK.md‘](./docs/MANUTENCAO_E_ROLLBACK.md)
- Funcoes-chave: [‘docs/FUNCOES_CHAVE.md‘](./docs/FUNCOES_CHAVE.md)
- Mapa de arquivos: [‘docs/MAPA_DE_ARQUIVOS.md‘](./docs/MAPA_DE_ARQUIVOS.md)

Secao consolidada: PROJECT_SAVEPOINT_VERSAO_01.md

PROJECT SAVEPOINT - VERSAO 01

Identificacao

- Nome: Versao 01
- Tag: ‘versao-01‘
- Commit base: ‘68e51ce‘
- Data: 2026-03-08

Estado funcional incluido

- app web React + Vite organizada em arquitetura profissional;
- area publica, auth, cliente e admin;
- Firebase Authentication + Firestore ativos;
- aprovacao de clientes por administrador;
- verificacao de e-mail exigida para login de cliente;
- fluxo de esqueci minha senha;
- menu mobile por icones e responsividade melhorada;
- backup JSON exportavel via interface.

Comandos para abrir exatamente esta versao

```
git fetch --all --tags
git switch -c abrir-versao-01 versao-01
npm.cmd install
npm.cmd run dev
```

Comandos para restaurar main para esta versao

```
git switch main
git pull
git switch -c backup-antes-versao-01
git switch main
git reset --hard versao-01
git push --force-with-lease origin main
```

Documentacao completa

Ver pasta [‘docs/‘](./docs/README.md).

Secao consolidada: docs/VERSAO_01.md

Versao 01 - Baseline Estavel

Este documento formaliza o marco de estabilidade solicitado por voce.

Identificacao da versao

- Nome da versao: ‘Versao 01‘
- Tag Git: ‘versao-01‘
- Commit de referencia: ‘68e51ce‘
- Data de criacao da tag: 2026-03-08

O que esta incluido na Versao 01

- arquitetura React organizada em camadas (‘components‘, ‘pages‘, ‘services‘, ‘hooks‘, ‘utils‘, ‘data‘);
- roteamento por area (publica, auth, cliente, admin);
- fluxo de cadastro/login com Firebase Authentication;
- fluxo de aprovacao de clientes por administrador;
- bloqueio de cliente pendente/reprovado;
- verificacao de e-mail para login de cliente;
- recuperacao de senha (esqueci minha senha);
- area admin com cadastro de clientes e aprovacao;
- backup manual por exportacao JSON;
- layout responsivo com menu mobile por icones;
- deploy automatico para GitHub Pages via GitHub Actions.

Como voltar para a Versao 01 (sem sobrescrever nada)

Use quando quiser abrir uma copia estavel para teste:

```
git fetch --all --tags
git switch -c teste-versao-01 versao-01
```

Isso cria um branch novo a partir da versao estavel.

Como restaurar o projeto principal para a Versao 01

Use quando voce quiser que o ‘main‘ volte ao estado da versao estavel.

Opcao segura (recomendada):

1. criar branch de backup do estado atual;
2. resetar localmente para a tag;
3. publicar com ‘--force-with-lease‘.

```
git switch main
git pull
git switch -c backup-antes-rollback
git switch main
git reset --hard versao-01
git push --force-with-lease origin main
```

Se quiser apenas desfazer uma mudanca ruim recente

Use ‘git revert‘ em vez de rollback total:

```
git switch main
git pull
git revert <hash-do-commit-ruim>
git push
```

Boas praticas para proximas versoes

- ao fechar cada marco estavel, crie nova tag (‘versao-02‘, ‘versao-03‘, etc.);
- atualize este documento com commit e funcionalidades;
- nunca faca alteracoes grandes sem commit intermediario.

Secao consolidada: docs/ARQUITETURA_E_FLUXOS.md

Arquitetura e Fluxos

Este documento explica como a aplicacao funciona por dentro.

1) Visao arquitetural

A aplicacao segue arquitetura de frontend com separacao por responsabilidade:

- ‘pages‘: telas de rota;

- ‘components‘: blocos reutilizaveis de UI;
- ‘hooks‘: estado global e regras de orquestracao;
- ‘services‘: regra de negocio, auth, persistencia, Firebase;
- ‘utils‘: funcoes utilitarias puras;
- ‘data‘: estado inicial do MVP local.

2) Fluxo de inicializacao (boot)

Quando o app abre:

1. ‘src/main.jsx‘ monta ‘HashRouter‘ + ‘AppStoreProvider‘ + ‘App‘.
2. ‘src/hooks/useAppStore.jsx‘ decide o modo:
   - ‘local‘: carrega ‘localStorage‘;
   - ‘firebase‘: usa sessao do Firebase Auth + Firestore em tempo real.
3. ‘src/App.jsx‘ renderiza rotas com guards:
   - publica (sem login),
   - guest (login/register),
   - cliente protegido,
   - admin protegido.

3) Rotas da aplicacao

Publicas

- ‘/‘ -> Landing page
- ‘/demo/kitchen‘ -> despensa demo
- ‘/demo/shopping‘ -> compras demo
- ‘/demo/recipes‘ -> receitas demo
- ‘/demo/tips‘ -> dicas/desafios demo
- aliases antigos com redirect:
  - ‘/demo/cozinha‘ -> ‘/demo/kitchen‘
  - ‘/demo/compras‘ -> ‘/demo/shopping‘
  - ‘/demo/receitas‘ -> ‘/demo/recipes‘
  - ‘/demo/dicas‘ -> ‘/demo/tips‘
- cadastro pendente:
  - ‘/register/pending‘
  - ‘/cadastro-pendente‘

Auth (somente para nao logado)

- ‘/login‘
- ‘/register‘
- ‘/forgot-password‘

Cliente (role client)

- ‘/app/dashboard‘
- ‘/app/inventory‘
- ‘/app/shopping‘
- ‘/app/recipes‘

- ‘/app/tips‘

Admin (role admin)

- ‘/admin/dashboard‘
- ‘/admin/clientes‘
- ‘/admin/cliente‘

4) Guardas de acesso

‘GuestRoute‘

- Se sessao existe e role for ‘admin‘: redireciona para ‘/admin/dashboard‘.
- Se sessao existe e role for ‘client‘: redireciona para ‘/app/dashboard‘.
- Se nao houver sessao: permite abrir login/register.

‘ProtectedRoute‘

- Sem sessao -> redireciona para ‘/login‘.
- Sessao com role fora da lista permitida -> redireciona para dashboard correto.
- Sessao valida -> renderiza ‘Outlet‘.

5) Estado global (‘useAppStore‘)

‘useAppStore‘ e o centro da aplicacao.

Responsabilidades principais:

- sessao atual (‘session‘);
- cliente ativo (‘activeClientId‘ / ‘activeClient‘);
- dados operacionais:
  - ‘inventories‘,
  - ‘shoppingLists‘,
  - ‘challenges‘;
- metrica de itens criticos (‘pendingCount‘);
- modos de backend (‘local‘ ou ‘firebase‘);
- funcoes publicas para paginas e componentes.

Funcoes expostas:

- auth:
  - ‘login‘
  - ‘register‘
  - ‘requestPasswordReset‘
  - ‘logout‘
- administracao:
  - ‘createClientByAdmin‘
  - ‘setClientApproval‘
  - ‘setAdminSelectedClientId‘
- operacao cliente:

  - ‘addInventory‘
  - ‘deleteInventory‘
  - ‘addShopping‘
  - ‘toggleShopping‘
  - ‘deleteShopping‘
  - ‘toggleChallenge‘
- utilitario:
  - ‘exportBackup‘

6) Modo local vs modo Firebase

Definido por ‘VITE_BACKEND_PROVIDER‘:

- ‘local‘: usa ‘LocalStorage‘ e dados de ‘defaultState‘.
- ‘firebase‘: usa Firebase Auth + Firestore.

Arquivo-chave: ‘src/services/backendAdapter.js‘.

7) Modelo de dados

Local (MVP)

Objeto unico com:

- ‘clientAccounts‘
- ‘adminAccounts‘
- ‘inventories‘ por cliente
- ‘shoppingLists‘ por cliente
- ‘challenges‘ por cliente

Firebase

Colecoes:

- ‘users/{uid}‘
  - ‘role‘
  - ‘approvalStatus‘
  - ‘name‘
  - ‘email‘
  - ‘businessType‘
- ‘clients/{uid}/inventory/{itemId}‘
- ‘clients/{uid}/shopping/{itemId}‘
- ‘clients/{uid}/meta/challenges‘

8) Fluxo de cadastro e aprovacao

1. Cliente preenche ‘/register‘.
2. ‘registerClientWithFirebase‘ cria usuario no Auth.
3. Sistema envia verificacao de e-mail.
4. Perfil e salvo em ‘users/{uid}‘ com:

  - ‘role=client‘
  - ‘approvalStatus=pending‘
5. Usuario vai para pagina de cadastro pendente.
6. Admin aprova em ‘/admin/clientes‘.
7. Cliente so entra se:
  - ‘emailVerified=true‘
  - ‘approvalStatus=approved‘

9) Fluxo admin

Admin pode:

- aprovar/reprovar cadastros pendentes;
- cadastrar novos clientes ja aprovados;
- escolher cliente ativo no toolbar;
- operar despensa/compras/desafios no contexto do cliente selecionado.

10) Cozinha Inteligente (motor atual)

Arquivo: ‘src/services/kitchenService.js‘

Entradas:
- lista de itens de estoque.

Saidas:
- sugestoes de receitas e acoes;
- dicas operacionais;
- priorizacao por validade (com ‘daysUntil‘ em ‘src/utils/date.js‘).

11) UI e responsividade

- Design system leve com Tailwind utilitario.
- Layout principal:
  - desktop: sidebar lateral + header;
  - mobile: navegacao inferior por icones.
- Componentes de secao desacoplados por dominio:
  - dashboard,
  - inventory,
  - shopping,
  - recipes,
  - tips.

12) Pontos de extensao planejados

- trocar provider em ‘backendAdapter‘ para ‘supabase‘ no futuro;
- adicionar modulo de analytics;
- criar camada de testes (unitarios e e2e);
- implementar funcoes server-side para logica critica (ex: aprovacao mais robusta).

Secao consolidada: docs/AMBIENTE_WINDOWS.md

Ambiente Windows - Passo a Passo Completo

Este guia e para rodar o projeto do zero em um computador Windows.

1) O que precisa instalar

Node.js

Para que serve:
- executa ferramentas JavaScript no computador;
- roda Vite (servidor local e build);
- instala dependencias com npm.

Versao recomendada neste projeto:
- Node >= 20
- npm >= 10

Como verificar:

```
node -v
npm.cmd -v
```

Git

Para que serve:
- controlar historico de codigo;
- salvar versoes;
- enviar projeto para GitHub.

Como verificar:

```
git --version
```

PowerShell

Ja vem no Windows. Sera usado para os comandos.

2) Problema comum no Windows (Execution Policy)

Erro comum:

- ‘npm : ... npm.ps1 nao pode ser carregado porque a execucao de scripts foi desabilitada‘

Solucao pratica:

- usar ‘npm.cmd‘ no lugar de ‘npm‘.

Exemplo:

```
npm.cmd install
npm.cmd run dev
```

3) Abrir o projeto

```
cd "C:\Users\emano\OneDrive\Documentos\Projetos Codex"
```

4) Instalar dependencias

```
npm.cmd install
```

Isso instala tudo de ‘package.json‘ na pasta ‘node_modules‘.

5) Rodar localmente

```
npm.cmd run dev -- --clearScreen false
```

Saida esperada:
- URL local, normalmente ‘http://localhost:5173/‘

Abra no navegador e teste a aplicacao.

6) Build de producao

```
npm.cmd run build
```

Gera a pasta ‘dist/‘ com os arquivos finais do site.

7) Preview local da build

```
npm.cmd run preview
```

Serve para validar a versao otimizada antes do deploy.

8) Estrutura local importante

- ‘node_modules/‘: bibliotecas instaladas (nao commitar)
- ‘dist/‘: build final (nao commitar)
- ‘.env.local‘: chaves locais (nao commitar)

9) Comandos Git basicos

Salvar alteracoes:

```
git add .
git commit -m "mensagem clara da mudanca"
git push
```

Ver estado:

```
git status
git log --oneline -n 10
```

10) Erros comuns e correcoes

Nao aparece nada apos ‘npm.cmd run dev‘

- pressione ‘Enter‘ no terminal;
- rode de novo com ‘--clearScreen false‘;
- confirme se esta na pasta correta do projeto.

Porta ocupada

- feche outro processo que usa 5173;
- ou rode com outra porta:

```
npm.cmd run dev -- --port 5174
```

Dependencia nao encontrada

```
npm.cmd install
```

Se persistir:

```
Remove-Item -Recurse -Force node_modules
Remove-Item -Force package-lock.json
npm.cmd install
```

11) Checklist rapido diario

1. ‘git pull‘
2. ‘npm.cmd install‘ (se mudou package-lock)
3. ‘npm.cmd run dev‘
4. desenvolver
5. ‘npm.cmd run build‘
6. ‘git add/commit/push‘

Secao consolidada: docs/FIREBASE_OPERACAO.md

Firebase - Operacao, Seguranca e Fluxos

Este guia explica como usar o Firebase neste projeto em nivel operacional.

1) Produtos Firebase usados

- **Authentication**: login por e-mail/senha, verificacao de e-mail e reset de senha.
- **Cloud Firestore**: banco NoSQL para usuarios, estoque, compras e desafios.

Nao usado no momento:

- Firebase Hosting (deploy atual esta no GitHub Pages)
- Cloud Functions
- Storage

2) Pre-requisitos

- projeto Firebase criado (ex: ‘desperdicio-zero-4fa38‘)
- plano Spark (custo zero inicial)
- app web registrado no Firebase

3) Habilitar Authentication

No Firebase Console:

1. ‘Authentication‘
2. ‘Get started‘
3. aba ‘Sign-in method‘
4. habilite ‘Email/Password‘
5. salve

4) Dominios autorizados no Auth

Em ‘Authentication‘ -> ‘Settings‘ -> ‘Authorized domains‘:

- ‘localhost‘
- ‘emanoelmagalhaes.github.io‘

Sem isso, podem ocorrer erros de dominio nao autorizado.

5) Criar Firestore

1. ‘Firestore Database‘
2. ‘Create database‘
3. edicao ‘Standard‘
4. escolha local (recomendado manter um unico local por projeto)

6) Publicar regras do Firestore

Arquivo usado no repositorio: [‘firestore.rules‘](../firestore.rules)

No console:

1. Firestore -> ‘Rules‘
2. apague tudo
3. cole o conteudo do arquivo
4. clique ‘Publish‘

Regra atual (referencia):

```
rules_version = ’2’;
service cloud.firestore {
```

```
  match /databases/{database}/documents {
    function isSignedIn() {
      return request.auth != null;
    }

    function isOwner(uid) {
      return isSignedIn() && request.auth.uid == uid;
    }

    function userDoc(uid) {
      return get(/databases/$(database)/documents/users/$(uid));
    }

    function userRole() {
      return userDoc(request.auth.uid).data.role;
    }

    function isAdmin() {
      return isSignedIn() && userRole() == 'admin';
    }

    function isApprovedClient(uid) {
      return isOwner(uid) && userDoc(uid).data.approvalStatus == 'approved';
    }

    function isClientSelfRegistration(uid) {
      return isOwner(uid)
        && request.resource.data.role == 'client'
        && request.resource.data.approvalStatus == 'pending';
    }

    function isOwnerSafeProfileUpdate(uid) {
      return isOwner(uid)
        && request.resource.data.role == resource.data.role
        && request.resource.data.approvalStatus == resource.data.approvalStatus;
    }

    match /users/{uid} {
      allow create: if isClientSelfRegistration(uid) || isAdmin();
      allow read: if isOwner(uid) || isAdmin();
      allow update: if isAdmin() || isOwnerSafeProfileUpdate(uid);
      allow delete: if false;
    }

    match /clients/{clientId} {
      allow read, write: if isApprovedClient(clientId) || isAdmin();

      match /{document=**} {
        allow read, write: if isApprovedClient(clientId) || isAdmin();
      }
    }

    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

7) Variaveis de ambiente locais

Arquivo local (nao commitar): ‘.env.local‘

Modelo (igual ao ‘.env.example‘):

```
VITE_BACKEND_PROVIDER=firebase
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
```

8) Variaveis no GitHub para deploy

No repo GitHub:

- ‘Settings‘ -> ‘Secrets and variables‘ -> ‘Actions‘

Variables

- ‘VITE_BACKEND_PROVIDER=firebase‘

Secrets

- ‘VITE_FIREBASE_API_KEY‘
- ‘VITE_FIREBASE_AUTH_DOMAIN‘
- ‘VITE_FIREBASE_PROJECT_ID‘
- ‘VITE_FIREBASE_STORAGE_BUCKET‘
- ‘VITE_FIREBASE_MESSAGING_SENDER_ID‘
- ‘VITE_FIREBASE_APP_ID‘

9) Estrutura de dados no Firestore

Colecao ‘users‘

Documento por UID do Auth:

- ‘role‘: ‘client‘ ou ‘admin‘
- ‘approvalStatus‘: ‘pending‘, ‘approved‘, ‘rejected‘
- ‘name‘
- ‘email‘
- ‘businessType‘
- metadados (‘createdAt‘, ‘approvedAt‘, etc.)

Colecao ‘clients‘

Subcolecoes por cliente:

- ‘clients/{uid}/inventory/{itemId}‘
- ‘clients/{uid}/shopping/{itemId}‘
- ‘clients/{uid}/meta/challenges‘

10) Promover usuario para admin

1. Firestore -> ‘Dados‘
2. colecao ‘users‘
3. abra o doc do usuario (UID)
4. ajuste campos:
   - ‘role = admin‘
   - ‘approvalStatus = approved‘
5. salvar

Depois, esse usuario consegue logar como admin.

11) Fluxo de cliente pendente

- cadastro cria usuario com ‘approvalStatus = pending‘;
- cliente recebe e-mail de verificacao;
- cliente NAO entra enquanto pendente;
- admin aprova em ‘/admin/clientes‘;
- cliente aprovado entra normalmente.

12) Erros comuns

‘Permissao negada no Firestore‘

Causas comuns:
- regras nao publicadas;
- documento ‘users/{uid}‘ sem ‘role/approvalStatus‘ esperados;
- usuario sem permissao para escrever no caminho.

‘Esse e-mail ja esta em uso‘

Pode ocorrer quando:
- usuario ja existe no Auth;
- houve tentativa antiga incompleta.

O sistema atual tenta recuperar cadastro pendente quando possivel.

Cliente cadastra com e-mail invalido

O Firebase Auth valida formato, mas nao confirma existencia real de caixa postal.
A confianca real vem da verificacao por link no e-mail (‘emailVerified‘).

13) Custo atual

Plano Spark costuma ser suficiente para fase inicial de testes.
Mesmo assim monitore:

- Firestore reads/writes
- Authentication MAU

Se crescer, avaliar upgrade para Blaze.

14) Arquivos de codigo relacionados

- ‘src/services/firebaseClient.js‘
- ‘src/services/firebaseAuthService.js‘
- ‘src/services/firebaseDataService.js‘
- ‘src/hooks/useAppStore.jsx‘
- ‘firestore.rules‘
- ‘.env.example‘

Secao consolidada: docs/DEPLOY_GITHUB_PAGES.md

Deploy com GitHub Pages - Passo a Passo

Este projeto esta configurado para deploy automatico via GitHub Actions.

1) O que ja esta pronto no codigo

Build Vite para Pages

Arquivo: ‘vite.config.js‘

- ‘base: ’./’‘
- combinado com ‘HashRouter‘ em ‘src/main.jsx‘
- evita problemas de refresh/404 em subrotas do Pages

Workflow automatico

Arquivo: ‘.github/workflows/deploy-gh-pages.yml‘

Dispara em cada ‘push‘ no branch ‘main‘:

1. checkout
2. setup Node 20
3. ‘npm install‘
4. ‘npm run build‘
5. publica ‘dist/‘ no branch ‘gh-pages‘

2) Primeiro deploy

No seu computador:

```
cd "C:\Users\emano\OneDrive\Documentos\Projetos Codex"
git add .
git commit -m "Deploy inicial"
git push
```

3) Acompanhar deploy

No GitHub:

1. abrir repositorio
2. aba ‘Actions‘
3. abrir workflow ‘Deploy GitHub Pages‘
4. conferir se o status ficou verde (‘Success‘)

4) Ativar Pages

No GitHub:

1. ‘Settings‘
2. ‘Pages‘
3. Source: ‘Deploy from a branch‘
4. Branch: ‘gh-pages‘ e pasta ‘/ (root)‘
5. salvar

URL esperada:

- ‘https://EmanoelMagalhaes.github.io/desperdicio-zero/‘

5) Configurar variaveis para build Firebase

No repo:

- ‘Settings‘ -> ‘Secrets and variables‘ -> ‘Actions‘

Variables

- ‘VITE_BACKEND_PROVIDER‘ (‘firebase‘ para modo online, ‘local‘ para modo MVP local)

Secrets

- ‘VITE_FIREBASE_API_KEY‘
- ‘VITE_FIREBASE_AUTH_DOMAIN‘
- ‘VITE_FIREBASE_PROJECT_ID‘
- ‘VITE_FIREBASE_STORAGE_BUCKET‘
- ‘VITE_FIREBASE_MESSAGING_SENDER_ID‘
- ‘VITE_FIREBASE_APP_ID‘

Sem isso, build pode subir mas app nao conecta no Firebase.

6) Deploy manual alternativo

Se quiser publicar manualmente pelo terminal:

```
npm.cmd install
npm.cmd run deploy
```

Isso usa ‘gh-pages -d dist‘ (script em ‘package.json‘).

7) Como validar depois de publicar

Checklist minimo:

1. home abre
2. cadastro de cliente funciona
3. login admin funciona
4. aprovacao de cliente funciona
5. menus mobile aparecem no celular
6. console do navegador sem erro critico

8) Problemas comuns

Workflow nao aparece

- confirme se arquivo existe em ‘.github/workflows/deploy-gh-pages.yml‘
- confirme push no branch ‘main‘

Workflow falha em build

- abra logs do job
- valide secrets e variables
- rode ‘npm.cmd run build‘ localmente para reproduzir

Site antigo em cache

- force reload (Ctrl+F5)
- abra em aba anonima

9) Fluxo padrao recomendado para toda mudanca

1. alterar codigo
2. ‘npm.cmd run build‘
3. ‘git add .‘
4. ‘git commit -m "..."‘
5. ‘git push‘
6. verificar ‘Actions‘
7. validar no URL publico

Secao consolidada: docs/MANUTENCAO_E_ROLLBACK.md

Manutencao e Rollback

Guia pratico para manter o projeto estavel sem perder historico.

1) Estrategia de versao

- baseline atual: ‘versao-01‘
- proxima recomendada: tag ‘versao-02‘ quando fechar novo ciclo estavel

2) Fluxo seguro para alterar codigo

1. atualizar main:

```
git switch main
git pull
```

2. criar branch de trabalho:

```
git switch -c codex/nome-da-mudanca
```

3. desenvolver e validar:

```
npm.cmd run dev
npm.cmd run build
```

4. commitar:

```
git add .
git commit -m "descricao objetiva"
```

5. publicar branch ou mergear no main.

3) Como criar uma nova versao estavel

```
git switch main
git pull
git tag -a versao-02 -m "Versao 02 - descricao"
git push origin versao-02
```

4) Como voltar para versao anterior

Caso A: desfazer 1 commit ruim sem destruir historico

```
git switch main
git pull
git revert <hash>
git push
```

Caso B: restaurar estado inteiro da tag

```
git switch main
git pull
git switch -c backup-antes-rollback
git switch main
git reset --hard versao-01
git push --force-with-lease origin main
```

Use apenas quando necessario e com backup.

5) Como recuperar alteracao local perdida

Ver reflog:

```
git reflog
```

Voltar para um ponto:

```
git switch -c resgate-<data> <hash-do-reflog>
```

6) Backup de dados da aplicacao

Backup funcional no app

No painel logado, usar botao ‘Exportar backup‘.

Arquivo gerado:
- JSON com inventario, compras, desafios e metadados do cliente ativo.

Backup Firebase (manual)

- export nativo do Firestore pode exigir GCP tooling;
- para fase atual, manter backups JSON periodicos ja reduz risco.

7) Checklist antes de cada deploy

1. ‘npm.cmd run build‘ sem erro
2. telas criticas testadas (login/register/admin)
3. regras Firestore atualizadas se houve mudanca de permissao
4. commit claro
5. push
6. workflow verde no GitHub Actions

8) Checklist apos deploy

1. abrir URL publica
2. testar login cliente
3. testar login admin
4. testar aprovacao de cliente
5. testar fluxo mobile rapido
6. checar console do navegador

9) Convencao de commit recomendada

Padrao simples:

- ‘feat: ...‘ nova funcionalidade
- ‘fix: ...‘ correcao
- ‘refactor: ...‘ reorganizacao
- ‘docs: ...‘ documentacao

10) Quando envolver outro desenvolvedor

Passe no handoff:

- link do repo
- link desta pasta ‘docs/‘
- tag estavel atual
- regras Firestore em uso
- lista de segredos necessarios no GitHub

Secao consolidada: docs/FUNCOES_CHAVE.md

Funcoes Chave e Onde Mexer

Este guia mapeia os pontos de codigo mais importantes para manutencao.

1) Entrada da aplicacao

‘src/main.jsx‘

Responsavel por montar:

- ‘HashRouter‘ (roteamento para GitHub Pages)
- ‘AppStoreProvider‘ (estado global)
- ‘App‘ (rotas)

Quando editar:
- se mudar tipo de router;
- se adicionar provider global (tema, i18n, query client etc.).

‘src/App.jsx‘

Define todas as rotas.

Onde editar:
- nova pagina publica: adicionar em bloco ‘PublicLayout‘;
- nova pagina de cliente: adicionar em bloco ‘ProtectedRoute allowRoles=[’client’]‘;
- nova pagina admin: adicionar em bloco ‘ProtectedRoute allowRoles=[’admin’]‘.

2) Estado global e orquestracao

‘src/hooks/useAppStore.jsx‘

Arquivo mais importante do projeto.

Estado principal

- ‘state‘: dados completos da aplicacao
- ‘session‘: usuario logado
- ‘ready‘: controle de carregamento
- ‘adminSelectedClientId‘: cliente ativo para admin

Funcoes de auth

- ‘login(mode, email, password)‘
- ‘register(form)‘
- ‘requestPasswordReset(email)‘
- ‘logout()‘

Funcoes admin

- ‘createClientByAdmin(form)‘
- ‘setClientApproval(clientId, approvalStatus)‘
- ‘setAdminSelectedClientId(id)‘

Funcoes operacionais

- ‘addInventory(item)‘
- ‘deleteInventory(id)‘
- ‘addShopping(item)‘
- ‘toggleShopping(id)‘
- ‘deleteShopping(id)‘
- ‘toggleChallenge(challenge)‘

Utilitarios de negocio

- ‘pendingCount‘ (itens criticos)
- ‘exportBackup()‘

Quando editar:
- qualquer regra global de sessao;
- sincronizacao local/Firebase;
- comportamento padrao de dados do app.

3) Servicos de backend e dados

‘src/services/backendAdapter.js‘

- define provider ativo (‘local‘, ‘firebase‘)
- funcoes:
  - ‘isRemoteEnabled()‘
  - ‘isFirebase()‘
  - ‘sync()‘ (stub para extensao futura)

‘src/services/authService.js‘ (modo local)

Funcoes:

- ‘loginWithMode(state, mode, email, password)‘
- ‘registerClientAccount(state, form)‘
- ‘createClientAccountByAdmin(state, form)‘

Usado quando ‘VITE_BACKEND_PROVIDER=local‘.

‘src/services/firebaseClient.js‘

Responsavel por:
- inicializar Firebase app;
- expor ‘auth‘ e ‘db‘;
- validar se as chaves existem (‘assertFirebaseReady‘);
- criar app secundario para fluxo sem derrubar sessao atual (‘createSecondaryAuthApp‘).

‘src/services/firebaseAuthService.js‘

Responsavel por auth real.

Funcoes principais:
- ‘loginWithFirebase(mode, email, password)‘
- ‘registerClientWithFirebase(form)‘
- ‘sendPasswordResetWithFirebase(email)‘
- ‘createClientByAdminWithFirebase(form, adminAccount)‘
- ‘subscribeAuthSession(onChange)‘
- ‘logoutFirebase()‘

Pontos criticos:
- verifica ‘emailVerified‘ para cliente;
- exige ‘approvalStatus=approved‘ para cliente entrar;
- trata erro de email ja em uso com fluxo de recuperacao.

‘src/services/firebaseDataService.js‘

Responsavel pelos dados operacionais no Firestore.

Funcoes principais:
- ‘loadClientData(clientId)‘
- ‘subscribeClientAccounts(onChange)‘
- ‘subscribeClientData(clientId, onChange)‘
- ‘updateClientApprovalStatus(clientId, status, adminId)‘
- ‘add/delete inventory‘
- ‘add/toggle/delete shopping‘
- ‘toggleChallengeItem(clientId, challenge)‘

‘src/services/storageService.js‘

Camada de LocalStorage:
- ‘loadState(storageKey, defaultState)‘
- ‘persistState(storageKey, state)‘

‘src/services/kitchenService.js‘

Motor da Cozinha Inteligente (versao atual baseada em regras).

Funcoes:
- ‘recipeSuggestions(items)‘
- ‘challengeTips()‘

4) Utilitarios

‘src/utils/date.js‘

- ‘addDays(days)‘
- ‘daysUntil(dateStr)‘
- ‘statusFromExpiry(dateStr)‘

Essencial para alertas de validade.

‘src/utils/ids.js‘

- ‘createId(prefix)‘

‘src/utils/export.js‘

- ‘downloadJson(filename, data)‘

Usado no botao de backup.

5) Componentes de layout

‘src/components/layout/PublicLayout.jsx‘

- header publico;
- navegacao desktop e mobile;
- links para login/cadastro.

‘src/components/layout/AppShell.jsx‘

Shell comum para cliente/admin:
- sidebar (desktop)
- header
- navegacao mobile inferior
- botoes sair e exportar backup

‘src/components/layout/ClientLayout.jsx‘

Injeta menu do cliente no ‘AppShell‘.

‘src/components/layout/AdminLayout.jsx‘

Injeta menu do admin no ‘AppShell‘.

6) Guardas

‘src/components/routing/GuestRoute.jsx‘

Bloqueia login/register para usuario ja autenticado.

‘src/components/routing/ProtectedRoute.jsx‘

Bloqueia rotas protegidas para nao autenticado ou role incorreto.

7) Componentes de secao

- ‘DashboardSection.jsx‘ -> visao geral e itens criticos
- ‘InventorySection.jsx‘ -> CRUD de estoque
- ‘ShoppingSection.jsx‘ -> CRUD de lista de compras
- ‘RecipesSection.jsx‘ -> sugestoes
- ‘TipsSection.jsx‘ -> dicas e desafios
- ‘AdminSummarySection.jsx‘ -> indicadores agregados do admin

Esses componentes concentram a maior parte da UI de negocio.

8) Paginas por dominio

Public

- ‘LandingPage‘
- ‘DemoKitchen‘
- ‘DemoShopping‘
- ‘DemoRecipes‘
- ‘DemoTips‘
- ‘PendingApprovalPage‘

Auth

- ‘LoginPage‘
- ‘RegisterPage‘
- ‘ForgotPasswordPage‘

Client

- ‘DashboardPage‘
- ‘InventoryPage‘
- ‘ShoppingPage‘
- ‘RecipesPage‘
- ‘TipsPage‘

Admin

- ‘AdminDashboard‘
- ‘ClientManager‘

- ‘ClientView‘

9) Onde mexer para tarefas comuns

Alterar texto de mensagem de login/cadastro

- ‘src/pages/auth/LoginPage.jsx‘
- ‘src/pages/auth/RegisterPage.jsx‘
- ‘src/services/firebaseAuthService.js‘

Mudar regra de aprovacao

- ‘src/services/firebaseAuthService.js‘
- ‘src/services/firebaseDataService.js‘
- ‘src/pages/admin/ClientManager.jsx‘
- ‘firestore.rules‘

Mudar sugestoes da Cozinha Inteligente

- ‘src/services/kitchenService.js‘

Mudar menu mobile

- ‘src/components/layout/AppShell.jsx‘
- ‘src/components/layout/PublicLayout.jsx‘

Mudar deploy

- ‘.github/workflows/deploy-gh-pages.yml‘
- ‘vite.config.js‘
- ‘package.json‘

10) Risco tecnico por arquivo

Alto risco (testar bem apos mexer):
- ‘src/hooks/useAppStore.jsx‘
- ‘src/services/firebaseAuthService.js‘
- ‘src/services/firebaseDataService.js‘
- ‘firestore.rules‘

Medio risco:
- ‘src/App.jsx‘
- ‘src/components/layout/AppShell.jsx‘
- ‘src/pages/admin/ClientManager.jsx‘

Baixo risco:
- arquivos de preview
- docs
- textos estaticos de paginas

Secao consolidada: docs/MAPA_DE_ARQUIVOS.md

Mapa Completo de Arquivos

Este documento lista os arquivos do repositorio e descreve funcao de cada um.

1) Raiz do projeto

Arquivo	Funcao
‘.env.example‘	Modelo das variaveis de ambiente usadas pelo Vite/Firebase.
‘.gitignore‘	Ignora ‘node_modules‘, ‘dist‘ e arquivos ‘.env*‘ (exceto ‘.env.example‘).
‘DESPERDICIO_ZERO_PROJECT_CONTEXT.md‘	Contexto macro do produto (visao, roadmap e escopo).
‘FIREBASE_SETUP.md‘	Guia resumido de setup Firebase e deploy com variaveis.
‘PROJECT_SAVEPOINT_2026-03-07.md‘	Savepoint intermediario do projeto.
‘PROJECT_SAVEPOINT_2026-03-07_FINAL.md‘	Savepoint final antigo com estado online de epoca.
‘README.md‘	Readme principal do repositorio com stack e comandos basicos.
‘firestore.rules‘	Regras de seguranca do Cloud Firestore.
‘index.html‘	HTML base carregado pelo Vite; injeta ‘src/main.jsx‘.
‘package.json‘	Scripts, dependencias e engines do projeto.
‘package-lock.json‘	Lockfile exato de dependencias npm.
‘postcss.config.js‘	Plugins PostCSS (Tailwind + Autoprefixer).
‘tailwind.config.js‘	Configuracao do Tailwind e paths de varredura.
‘vite.config.js‘	Config Vite com plugin React e ‘base: ’./’‘.

2) GitHub Actions

Arquivo	Funcao
‘.github/workflows/deploy-gh-pages.yml‘	Pipeline CI/CD que builda e publica no branch ‘gh-pages‘.

3) Previews

Arquivo	Funcao
‘previews/admin-dashboard-preview.png‘	Preview visual do dashboard admin (bitmap).
‘previews/admin-dashboard-preview.svg‘	Preview visual do dashboard admin (vetorial).
‘previews/client-dashboard-preview.png‘	Preview visual do dashboard cliente (bitmap).
‘previews/client-dashboard-preview.svg‘	Preview visual do dashboard cliente (vetorial).

| ‘previews/public-home-preview.png‘ | Preview visual da home publica (bitmap). |
| ‘previews/public-home-preview.svg‘ | Preview visual da home publica (vetorial). |

4) Entrada e estilo global (‘src‘)

Arquivo	Funcao
‘src/main.jsx‘	Ponto de entrada React, monta router e provider global.
‘src/App.jsx‘	Declaracao central de todas as rotas.
‘src/index.css‘	Estilos globais base com Tailwind e reset simples.

5) Dados e estado

Arquivo	Funcao
‘src/data/defaultState.js‘	Estado inicial do modo local (contas demo, estoque, compras, desafios).
‘src/hooks/useAppStore.jsx‘	Store global: sessao, dados operacionais, integracao local/Firebase, acoes da app.

6) Servicos

Arquivo	Funcao
‘src/services/backendAdapter.js‘	Define provider ativo (‘local‘/‘firebase‘) e ponto de extensao futuro.
‘src/services/storageService.js‘	Persistencia local via LocalStorage.
‘src/services/authService.js‘	Auth em modo local (sem backend real).
‘src/services/firebaseClient.js‘	Inicializacao Firebase app/auth/db e validacao de configuracao.
‘src/services/firebaseAuthService.js‘	Login/cadastro/reset/verificacao/aprovacao no Firebase Auth + perfil em Firestore.
‘src/services/firebaseDataService.js‘	Operacoes de dados no Firestore (inventory/shopping/challenges/users).
‘src/services/kitchenService.js‘	Regras da Cozinha Inteligente (sugestoes e dicas).

7) Utilitarios

Arquivo	Funcao
‘src/utils/date.js‘	Funcoes de data para validade e status de risco.
‘src/utils/export.js‘	Exportacao de backup JSON no navegador.
‘src/utils/ids.js‘	Gerador de IDs unicos simples com prefixo + timestamp.

8) Components - comuns

Arquivo	Funcao

‘src/components/common/LoadingScreen.jsx‘	Tela de carregamento inicial enquanto store prepara estado.
‘src/components/common/MetricCard.jsx‘	Card de metrica reutilizavel com icone e tema.
‘src/components/common/SectionTitle.jsx‘	Bloco padrao de titulo/subtitulo de secao.

9) Components - layout

Arquivo	Funcao
‘src/components/layout/PublicLayout.jsx‘	Layout publico com header e menu mobile.
‘src/components/layout/AppShell.jsx‘	Shell principal de cliente/admin com sidebar, header, feedback e menu mobile.
‘src/components/layout/ClientLayout.jsx‘	Configura menu do cliente e reaproveita ‘AppShell‘.
‘src/components/layout/AdminLayout.jsx‘	Configura menu do admin e reaproveita ‘AppShell‘.

10) Components - roteamento

Arquivo	Funcao
‘src/components/routing/GuestRoute.jsx‘	Bloqueia rotas de auth para usuario autenticado.
‘src/components/routing/ProtectedRoute.jsx‘	Protege rotas por autenticacao e role.

11) Components - admin

Arquivo	Funcao
‘src/components/admin/AdminClientToolbar.jsx‘	Seletor de cliente ativo para operacao admin em tempo real.

12) Components - secoes de negocio

Arquivo	Funcao
‘src/components/sections/AdminSummarySection.jsx‘	Indicadores agregados de clientes para admin.
‘src/components/sections/DashboardSection.jsx‘	Visao geral com itens criticos e atalhos operacionais.
‘src/components/sections/InventorySection.jsx‘	Formulario e lista de estoque com CRUD e feedback.
‘src/components/sections/ShoppingSection.jsx‘	Formulario e lista de compras com prioridade e check.
‘src/components/sections/RecipesSection.jsx‘	Cartoes de sugestoes de

receitas/acoes. |
| ‘src/components/sections/TipsSection.jsx‘ | Dicas praticas e desafios com controle de conclusao. |

13) Pages - publicas

Arquivo	Funcao
‘src/pages/public/LandingPage.jsx‘	Home publica com proposta de valor e atalhos de demo.
‘src/pages/public/DemoKitchen.jsx‘	Encapsula ‘InventorySection‘ em modo somente leitura.
‘src/pages/public/DemoShopping.jsx‘	Encapsula ‘ShoppingSection‘ em modo somente leitura.
‘src/pages/public/DemoRecipes.jsx‘	Encapsula ‘RecipesSection‘ para dados demo.
‘src/pages/public/DemoTips.jsx‘	Encapsula ‘TipsSection‘ em modo somente leitura.
‘src/pages/public/PendingApprovalPage.jsx‘	Tela apos cadastro pendente com orientacao e marketing.

14) Pages - auth

Arquivo	Funcao
‘src/pages/auth/LoginPage.jsx‘	Login cliente/admin com feedback e redirecionamento.
‘src/pages/auth/RegisterPage.jsx‘	Cadastro de cliente com redirecionamento para pendencia.
‘src/pages/auth/ForgotPasswordPage.jsx‘	Envio de reset de senha.

15) Pages - cliente

Arquivo	Funcao
‘src/pages/client/DashboardPage.jsx‘	Wrapper do dashboard cliente.
‘src/pages/client/InventoryPage.jsx‘	Wrapper da secao de estoque cliente.
‘src/pages/client/ShoppingPage.jsx‘	Wrapper da secao de compras cliente.
‘src/pages/client/RecipesPage.jsx‘	Wrapper da secao de receitas cliente.
‘src/pages/client/TipsPage.jsx‘	Wrapper da secao de dicas/desafios cliente.

16) Pages - admin

Arquivo	Funcao
‘src/pages/admin/AdminDashboard.jsx‘	Wrapper do resumo administrativo.
‘src/pages/admin/ClientManager.jsx‘	Aprovacao/reprovacao de clientes e cadastro por admin.
‘src/pages/admin/ClientView.jsx‘	Simulacao da visao operacional do cliente pelo admin.

17) Arquivos locais que normalmente NAO vao para Git

Caminho	Funcao
‘.env.local‘	Variaveis locais (segredos).
‘node_modules/‘	Dependencias instaladas localmente.
‘dist/‘	Build de producao gerada localmente.
‘.git/‘	Metadados internos do repositorio Git.

18) Sobre Node, Firebase, GitHub e o papel de cada um

- **Node.js**: runtime para executar ferramentas JS (Vite, npm scripts).
- **npm**: gerenciador de pacotes e scripts (‘install‘, ‘run dev‘, ‘run build‘).
- **Git**: controle de versao local (commits, branches, tags, rollback).
- **GitHub**: repositorio remoto e colaboracao.
- **GitHub Actions**: automacao de build/deploy.
- **GitHub Pages**: hospedagem estatica do front-end.
- **Firebase Authentication**: login, cadastro, verificacao de e-mail, reset de senha.
- **Cloud Firestore**: banco online com dados operacionais e regras de acesso.

19) Arquivos de documentacao adicionados na Versao 01

Arquivo	Funcao
‘PROJECT_SAVEPOINT_VERSAO_01.md‘	Savepoint oficial da Versao 01 com comandos de recuperacao.
‘docs/README.md‘	Indice principal da documentacao tecnica.
‘docs/VERSAO_01.md‘	Documento oficial da baseline estavel.
‘docs/ARQUITETURA_E_FLUXOS.md‘	Arquitetura tecnica, rotas e fluxos de negocio.
‘docs/AMBIENTE_WINDOWS.md‘	Instalacao e operacao local no Windows.
‘docs/FIREBASE_OPERACAO.md‘	Configuracao e operacao do Firebase.
‘docs/DEPLOY_GITHUB_PAGES.md‘	Processo completo de deploy no GitHub Pages.
‘docs/MANUTENCAO_E_ROLLBACK.md‘	Estrategias de manutencao, rollback e recuperacao.
‘docs/FUNCOES_CHAVE.md‘	Mapa de funcoes, responsabilidades e pontos de edicao.
‘docs/MAPA_DE_ARQUIVOS.md‘	Inventario detalhado dos arquivos do repositorio.

Apendice Tecnico - Codigos e Configuracoes

A partir daqui, voce encontra os principais arquivos tecnicos na integra para consulta, estudo e auditoria.

Arquivo: .env.example

```
VITE_BACKEND_PROVIDER=local
```

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
```

Arquivo: package.json

```
{
  "name": "desperdicio-zero",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "predeploy": "npm run build",
    "deploy": "gh-pages -d dist"
  },
  "engines": {
    "node": ">=20.0.0",
    "npm": ">=10.0.0"
  },
  "dependencies": {
    "firebase": "^12.10.0",
    "framer-motion": "^11.14.0",
    "lucide-react": "^0.468.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.28.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.3",
    "autoprefixer": "^10.4.20",
    "gh-pages": "^6.2.0",
    "postcss": "^8.4.49",
    "tailwindcss": "^3.4.14",
    "vite": "^5.4.10"
  }
}
```

Arquivo: vite.config.js

```
import { defineConfig } from ’vite’;
import react from ’@vitejs/plugin-react’;

export default defineConfig({
  plugins: [react()],
  base: ’./’,
});
```

Arquivo: tailwind.config.js

```
/** @type {import(’tailwindcss’).Config} */
export default {
  content: [’./index.html’, ’./src/**/*.{js,jsx}’],
```

```
  theme: {
    extend: {},
  },
  plugins: [],
};
```

Arquivo: postcss.config.js

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
};
```

Arquivo: .github/workflows/deploy-gh-pages.yml

```
name: Deploy GitHub Pages

on:
  push:
    branches:
      - main

permissions:
  contents: write

jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      VITE_BACKEND_PROVIDER: ${{ vars.VITE_BACKEND_PROVIDER }}
      VITE_FIREBASE_API_KEY: ${{ secrets.VITE_FIREBASE_API_KEY }}
      VITE_FIREBASE_AUTH_DOMAIN: ${{ secrets.VITE_FIREBASE_AUTH_DOMAIN }}
      VITE_FIREBASE_PROJECT_ID: ${{ secrets.VITE_FIREBASE_PROJECT_ID }}
      VITE_FIREBASE_STORAGE_BUCKET: ${{ secrets.VITE_FIREBASE_STORAGE_BUCKET }}
      VITE_FIREBASE_MESSAGING_SENDER_ID: ${{ secrets.VITE_FIREBASE_MESSAGING_SENDER_ID }}
      VITE_FIREBASE_APP_ID: ${{ secrets.VITE_FIREBASE_APP_ID }}

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install
        run: npm install

      - name: Build
        run: npm run build

      - name: Publish
        uses: peaceiris/actions-gh-pages@v4
        with:
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_dir: ./dist
```

Arquivo: firestore.rules

```
rules_version = ’2’;
service cloud.firestore {
  match /databases/{database}/documents {
    function isSignedIn() {
      return request.auth != null;
    }

    function isOwner(uid) {
      return isSignedIn() && request.auth.uid == uid;
    }

    function userDoc(uid) {
      return get(/databases/$(database)/documents/users/$(uid));
    }

    function userRole() {
      return userDoc(request.auth.uid).data.role;
    }

    function isAdmin() {
      return isSignedIn() && userRole() == ’admin’;
    }

    function isApprovedClient(uid) {
      return isOwner(uid) && userDoc(uid).data.approvalStatus == ’approved’;
    }

    function isClientSelfRegistration(uid) {
      return isOwner(uid)
        && request.resource.data.role == ’client’
        && request.resource.data.approvalStatus == ’pending’;
    }

    function isOwnerSafeProfileUpdate(uid) {
      return isOwner(uid)
        && request.resource.data.role == resource.data.role
        && request.resource.data.approvalStatus == resource.data.approvalStatus;
    }

    match /users/{uid} {
      allow create: if isClientSelfRegistration(uid) || isAdmin();
      allow read: if isOwner(uid) || isAdmin();
      allow update: if isAdmin() || isOwnerSafeProfileUpdate(uid);
      allow delete: if false;
    }

    match /clients/{clientId} {
      allow read, write: if isApprovedClient(clientId) || isAdmin();

      match /{document=**} {
        allow read, write: if isApprovedClient(clientId) || isAdmin();
      }
    }

    match /{document=**} {
      allow read, write: if false;
    }
  }
```

```
}
```

Arquivo: src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { HashRouter } from 'react-router-dom';
import App from './App';
import { AppStoreProvider } from './hooks/useAppStore';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <HashRouter>
      <AppStoreProvider>
        <App />
      </AppStoreProvider>
    </HashRouter>
  </React.StrictMode>
);
```

Arquivo: src/App.jsx

```
import { Navigate, Route, Routes } from 'react-router-dom';
import LoadingScreen from './components/common/LoadingScreen';
import AdminLayout from './components/layout/AdminLayout';
import ClientLayout from './components/layout/ClientLayout';
import PublicLayout from './components/layout/PublicLayout';
import GuestRoute from './components/routing/GuestRoute';
import ProtectedRoute from './components/routing/ProtectedRoute';
import { useAppStore } from './hooks/useAppStore';
import LoginPage from './pages/auth/LoginPage';
import RegisterPage from './pages/auth/RegisterPage';
import ForgotPasswordPage from './pages/auth/ForgotPasswordPage';
import AdminDashboard from './pages/admin/AdminDashboard';
import ClientManager from './pages/admin/ClientManager';
import ClientView from './pages/admin/ClientView';
import DashboardPage from './pages/client/DashboardPage';
import InventoryPage from './pages/client/InventoryPage';
import RecipesPage from './pages/client/RecipesPage';
import ShoppingPage from './pages/client/ShoppingPage';
import TipsPage from './pages/client/TipsPage';
import DemoKitchen from './pages/public/DemoKitchen';
import DemoRecipes from './pages/public/DemoRecipes';
import DemoShopping from './pages/public/DemoShopping';
import DemoTips from './pages/public/DemoTips';
import LandingPage from './pages/public/LandingPage';
import PendingApprovalPage from './pages/public/PendingApprovalPage';

export default function App() {
  const { ready } = useAppStore();

  if (!ready) {
    return <LoadingScreen />;
  }

  return (
    <Routes>
      <Route element={<PublicLayout />}>
```

```
        <Route index element={<LandingPage />} />
        <Route path="/demo/kitchen" element={<DemoKitchen />} />
        <Route path="/demo/recipes" element={<DemoRecipes />} />
        <Route path="/demo/shopping" element={<DemoShopping />} />
        <Route path="/demo/tips" element={<DemoTips />} />

        <Route path="/demo/cozinha" element={<Navigate to="/demo/kitchen" replace />} />
        <Route path="/demo/receitas" element={<Navigate to="/demo/recipes" replace />} />
        <Route path="/demo/compras" element={<Navigate to="/demo/shopping" replace />} />
        <Route path="/demo/dicas" element={<Navigate to="/demo/tips" replace />} />

        <Route path="/register/pending" element={<PendingApprovalPage />} />
        <Route path="/cadastro-pendente" element={<PendingApprovalPage />} />
      </Route>

      <Route element={<GuestRoute />}>
        <Route element={<PublicLayout />}>
          <Route path="/login" element={<LoginPage />} />
          <Route path="/register" element={<RegisterPage />} />
          <Route path="/forgot-password" element={<ForgotPasswordPage />} />
        </Route>
      </Route>

      <Route element={<ProtectedRoute allowRoles={['client']} />}>
        <Route element={<ClientLayout />}>
          <Route path="/app" element={<Navigate to="/app/dashboard" replace />} />
          <Route path="/app/dashboard" element={<DashboardPage />} />
          <Route path="/app/inventory" element={<InventoryPage />} />
          <Route path="/app/shopping" element={<ShoppingPage />} />
          <Route path="/app/recipes" element={<RecipesPage />} />
          <Route path="/app/tips" element={<TipsPage />} />
        </Route>
      </Route>

      <Route element={<ProtectedRoute allowRoles={['admin']} />}>
        <Route element={<AdminLayout />}>
          <Route path="/admin" element={<Navigate to="/admin/dashboard" replace />} />
          <Route path="/admin/dashboard" element={<AdminDashboard />} />
          <Route path="/admin/clientes" element={<ClientManager />} />
          <Route path="/admin/cliente" element={<ClientView />} />
        </Route>
      </Route>

      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  );
}
```

Arquivo: src/hooks/useAppStore.jsx

```
import { createContext, useCallback, useContext, useEffect, useMemo, useState } from
'react';
import { STORAGE_KEY, defaultState } from '../data/defaultState';
import { createClientAccountByAdmin, loginWithMode, registerClientAccount } from
'../services/authService';
import { createClientByAdminWithFirebase, loginWithFirebase, logoutFirebase,
registerClientWithFirebase, sendPasswordResetWithFirebase, subscribeAuthSession } from
'../services/firebaseAuthService';
import {
  addInventoryItem,
```

```
  addShoppingItem,
  deleteInventoryItem,
  deleteShoppingItem,
  loadClientData,
  subscribeClientAccounts,
  subscribeClientData,
  updateClientApprovalStatus,
  toggleChallengeItem,
  toggleShoppingItem,
} from '../services/firebaseDataService';
import { backendAdapter } from '../services/backendAdapter';
import { loadState, persistState } from '../services/storageService';
import { daysUntil } from '../utils/date';
import { downloadJson } from '../utils/export';
import { createId } from '../utils/ids';

const AppStoreContext = createContext(null);

function upsertClientAccount(accounts, account) {
  if (!account || account.role !== 'client') return accounts;

  const exists = accounts.some((item) => item.id === account.id);
  if (exists) {
    return accounts.map((item) => (item.id === account.id ? { ...item, ...account } :
item));
  }

  return [...accounts, account];
}

function createEmptyOperationalState() {
  return {
    clientAccounts: [],
    adminAccounts: [],
    inventories: {},
    shoppingLists: {},
    challenges: {},
  };
}

export function AppStoreProvider({ children }) {
  const firebaseMode = backendAdapter.isFirebase();

  const [state, setState] = useState(defaultState);
  const [session, setSession] = useState(null);
  const [adminSelectedClientId, setAdminSelectedClientId] = useState('');
  const [ready, setReady] = useState(false);

  useEffect(() => {
    if (firebaseMode) {
      setState((prev) => ({ ...prev, ...createEmptyOperationalState() }));
      setReady(true);
      return;
    }

    const loaded = loadState(STORAGE_KEY, defaultState);
    setState(loaded);
    setAdminSelectedClientId('cliente-demo');
    setReady(true);
  }, [firebaseMode]);
```

```
useEffect(() => {
  if (!ready || firebaseMode) return;
  persistState(STORAGE_KEY, state);
}, [ready, state, firebaseMode]);

useEffect(() => {
  if (!firebaseMode) return;

  const unsubscribe = subscribeAuthSession((account) => {
    if (!account) {
      setSession(null);
      setState((prev) => ({ ...prev, ...createEmptyOperationalState() }));
      setAdminSelectedClientId('');
      return;
    }

    setSession(account);
    if (account.role === 'client') {
      setState((prev) => ({
        ...prev,
        clientAccounts: upsertClientAccount(prev.clientAccounts, account),
      }));
    }
  });

  return unsubscribe;
}, [firebaseMode]);

useEffect(() => {
  if (!firebaseMode || session?.role !== 'client') return;

  const clientId = session.id;
  const unsubscribe = subscribeClientData(clientId, (data) => {
    setState((prev) => ({
      ...prev,
      clientAccounts: upsertClientAccount(prev.clientAccounts, session),
      inventories: { ...prev.inventories, [clientId]: data.inventory },
      shoppingLists: { ...prev.shoppingLists, [clientId]: data.shoppingList },
      challenges: { ...prev.challenges, [clientId]: data.challenges },
    }));
  });

  return unsubscribe;
}, [firebaseMode, session]);

useEffect(() => {
  if (!firebaseMode || session?.role !== 'admin') return;

  let active = true;

  const unsubscribe = subscribeClientAccounts(async (clients) => {
    if (!active) return;

    setState((prev) => ({ ...prev, clientAccounts: clients }));
    setAdminSelectedClientId((prev) => {
      if (prev && clients.some((client) => client.id === prev)) return prev;
      return clients[0]?.id || '';
    });

    const loaded = await Promise.all(
      clients.map(async (client) => ({
```

```
          id: client.id,
          data: await loadClientData(client.id),
        }))
      );

      if (!active) return;

      setState((prev) => {
        const inventories = { ...prev.inventories };
        const shoppingLists = { ...prev.shoppingLists };
        const challenges = { ...prev.challenges };

        loaded.forEach(({ id, data }) => {
          inventories[id] = data.inventory;
          shoppingLists[id] = data.shoppingList;
          challenges[id] = data.challenges;
        });

        return {
          ...prev,
          inventories,
          shoppingLists,
          challenges,
        };
      });
    });

    return () => {
      active = false;
      unsubscribe();
    };
  }, [firebaseMode, session?.role]);

  useEffect(() => {
    if (!firebaseMode || session?.role !== 'admin' || !adminSelectedClientId) return;

    const unsubscribe = subscribeClientData(adminSelectedClientId, (data) => {
      setState((prev) => ({
        ...prev,
        inventories: { ...prev.inventories, [adminSelectedClientId]: data.inventory },
        shoppingLists: { ...prev.shoppingLists, [adminSelectedClientId]: data.shoppingList
},
        challenges: { ...prev.challenges, [adminSelectedClientId]: data.challenges },
      }));
    });

    return unsubscribe;
  }, [firebaseMode, session?.role, adminSelectedClientId]);

  useEffect(() => {
    if (session?.role !== 'admin') return;

    const exists = state.clientAccounts.some((client) => client.id ===
adminSelectedClientId);
    if (!exists) {
      setAdminSelectedClientId(state.clientAccounts[0]?.id || '');
    }
  }, [session, state.clientAccounts, adminSelectedClientId]);

  const activeClientId = useMemo(() => {
    if (session?.role === 'client') return session.id;
```

```
    if (session?.role === 'admin') return adminSelectedClientId ||
state.clientAccounts[0]?.id || '';
    return '';
  }, [session, adminSelectedClientId, state.clientAccounts]);

  const activeClient = useMemo(
    () => state.clientAccounts.find((client) => client.id === activeClientId) || null,
    [state.clientAccounts, activeClientId]
  );

  const inventory = useMemo(() => state.inventories[activeClientId] || [],
[state.inventories, activeClientId]);
  const shoppingList = useMemo(() => state.shoppingLists[activeClientId] || [],
[state.shoppingLists, activeClientId]);
  const challengeState = useMemo(
    () => state.challenges[activeClientId] || { completed: [], current: [] },
    [state.challenges, activeClientId]
  );

  const demoInventory = defaultState.inventories['cliente-demo'] || [];
  const demoShoppingList = defaultState.shoppingLists['cliente-demo'] || [];
  const demoChallenges = defaultState.challenges['cliente-demo'] || { completed: [],
current: [] };

  const pendingCount = useMemo(() => {
    if (!session) return 0;

    if (session.role === 'client') {
      return inventory.filter((item) => daysUntil(item.expiry) <= 2).length;
    }

    return Object.values(state.inventories)
      .flat()
      .filter((item) => daysUntil(item.expiry) <= 2).length;
  }, [session, state.inventories, inventory]);

  const login = useCallback(
    async (mode, email, password) => {
      if (firebaseMode) {
        const result = await loginWithFirebase(mode, email, password);
        if (!result.ok) return result;

        setSession(result.account);

        if (result.account.role === 'admin') {
          setAdminSelectedClientId((prev) => prev || state.clientAccounts[0]?.id || '');
        }

        return result;
      }

      const result = loginWithMode(state, mode, email, password);
      if (!result.ok) return result;

      setSession(result.account);
      if (result.account.role === 'admin') {
        setAdminSelectedClientId(state.clientAccounts[0]?.id || '');
      }

      return { ok: true, account: result.account };
    },
```

```
    [firebaseMode, state]
  );

  const register = useCallback(
    async (form) => {
      if (firebaseMode) {
        return registerClientWithFirebase(form);
      }

      const result = registerClientAccount(state, form);
      if (!result.ok) return result;

      setState(result.nextState);
      return result;
    },
    [firebaseMode, state]
  );
  const requestPasswordReset = useCallback(
    async (email) => {
      if (!email) {
        return { ok: false, error: 'Informe o e-mail para recuperar a senha.' };
      }

      if (firebaseMode) {
        return sendPasswordResetWithFirebase(email);
      }

      return {
        ok: true,
        message:
          'No modo local nao ha envio real de e-mail. Em producao com Firebase, o link de
redefinicao sera enviado para o endereco informado.',
      };
    },
    [firebaseMode]
  );
  const createClientByAdmin = useCallback(
    async (form) => {
      if (session?.role !== 'admin') {
        return { ok: false, error: 'Apenas administradores podem cadastrar clientes.' };
      }

      if (firebaseMode) {
        const result = await createClientByAdminWithFirebase(form, session);
        if (!result.ok) return result;

        setState((prev) => ({
          ...prev,
          clientAccounts: upsertClientAccount(prev.clientAccounts, result.account),
          inventories: {
            ...prev.inventories,
            [result.account.id]: prev.inventories[result.account.id] || [],
          },
          shoppingLists: {
            ...prev.shoppingLists,
            [result.account.id]: prev.shoppingLists[result.account.id] || [],
          },
          challenges: {
            ...prev.challenges,
            [result.account.id]: prev.challenges[result.account.id] || { completed: [],
current: [] },
```

```
        },
      }));

      return result;
    }

    const result = createClientAccountByAdmin(state, form);
    if (!result.ok) return result;

    setState(result.nextState);

    return { ok: true, account: result.account };
  },
  [firebaseMode, session, state]
);

const setClientApproval = useCallback(
  async (clientId, approvalStatus) => {
    if (!clientId) {
      return { ok: false, error: 'Cliente invalido.' };
    }

    if (session?.role !== 'admin') {
      return { ok: false, error: 'Apenas administradores podem aprovar clientes.' };
    }

    if (firebaseMode) {
      await updateClientApprovalStatus(clientId, approvalStatus, session.id);
    }

    setState((prev) => ({
      ...prev,
      clientAccounts: prev.clientAccounts.map((client) =>
        client.id === clientId ? { ...client, approvalStatus } : client
      ),
    }));

    return { ok: true };
  },
  [firebaseMode, session]
);

const logout = useCallback(async () => {
  if (firebaseMode) {
    await logoutFirebase();
  }

  setSession(null);
}, [firebaseMode]);

const addInventory = useCallback(
  async (item, clientId = activeClientId) => {
    if (!clientId) return;

    const payload = { ...item, id: item.id || createId('item') };

    if (firebaseMode) {
      await addInventoryItem(clientId, payload);
    }

    setState((prev) => ({
```

```
        ...prev,
        inventories: {
          ...prev.inventories,
          [clientId]: [...(prev.inventories[clientId] || []), payload],
        },
      }));
    },
    [activeClientId, firebaseMode]
  );

  const deleteInventory = useCallback(
    async (id, clientId = activeClientId) => {
      if (!clientId) return;

      if (firebaseMode) {
        await deleteInventoryItem(clientId, id);
      }

      setState((prev) => ({
        ...prev,
        inventories: {
          ...prev.inventories,
          [clientId]: (prev.inventories[clientId] || []).filter((item) => item.id !== id),
        },
      }));
    },
    [activeClientId, firebaseMode]
  );

  const addShopping = useCallback(
    async (item, clientId = activeClientId) => {
      if (!clientId) return;

      const payload = { ...item, id: item.id || createId('shop'), checked:
Boolean(item.checked) };

      if (firebaseMode) {
        await addShoppingItem(clientId, payload);
      }

      setState((prev) => ({
        ...prev,
        shoppingLists: {
          ...prev.shoppingLists,
          [clientId]: [...(prev.shoppingLists[clientId] || []), payload],
        },
      }));
    },
    [activeClientId, firebaseMode]
  );

  const toggleShopping = useCallback(
    async (id, clientId = activeClientId) => {
      if (!clientId) return;

      if (firebaseMode) {
        await toggleShoppingItem(clientId, id);
      }

      setState((prev) => ({
        ...prev,
```

```
        shoppingLists: {
          ...prev.shoppingLists,
          [clientId]: (prev.shoppingLists[clientId] || []).map((item) =>
            item.id === id ? { ...item, checked: !item.checked } : item
          ),
        },
      }));
    },
    [activeClientId, firebaseMode]
  );

  const deleteShopping = useCallback(
    async (id, clientId = activeClientId) => {
      if (!clientId) return;

      if (firebaseMode) {
        await deleteShoppingItem(clientId, id);
      }

      setState((prev) => ({
        ...prev,
        shoppingLists: {
          ...prev.shoppingLists,
          [clientId]: (prev.shoppingLists[clientId] || []).filter((item) => item.id !==
id),
        },
      }));
    },
    [activeClientId, firebaseMode]
  );

  const toggleChallenge = useCallback(
    async (challenge, clientId = activeClientId) => {
      if (!clientId) return;

      if (firebaseMode) {
        const nextChallenges = await toggleChallengeItem(clientId, challenge);

        setState((prev) => ({
          ...prev,
          challenges: {
            ...prev.challenges,
            [clientId]: nextChallenges,
          },
        }));

        return;
      }

      setState((prev) => {
        const current = prev.challenges[clientId] || { completed: [], current: [] };
        const isDone = current.completed.includes(challenge);

        return {
          ...prev,
          challenges: {
            ...prev.challenges,
            [clientId]: {
              ...current,
              completed: isDone
                ? current.completed.filter((item) => item !== challenge)
```

```
                : [...current.completed, challenge],
            },
          },
        };
      });
    },
    [activeClientId, firebaseMode]
  );

  const exportBackup = useCallback(() => {
    if (!session || !activeClientId) {
      return { ok: false, error: 'Nenhum cliente ativo para exportacao.' };
    }

    const payload = {
      generatedAt: new Date().toISOString(),
      app: 'Desperdicio Zero',
      backendMode: firebaseMode ? 'firebase' : 'local',
      exportedBy: {
        id: session.id,
        role: session.role,
        email: session.email,
        name: session.name,
      },
      client: activeClient || {
        id: activeClientId,
        role: 'client',
      },
      data: {
        inventory: state.inventories[activeClientId] || [],
        shoppingList: state.shoppingLists[activeClientId] || [],
        challenges: state.challenges[activeClientId] || { completed: [], current: [] },
      },
    };

    const stamp = new Date().toISOString().replace(/[:.]/g, '-');
    const fileName = 'backup-${activeClientId}-${stamp}.json';

    downloadJson(fileName, payload);

    return { ok: true, fileName };
  }, [session, activeClientId, activeClient, firebaseMode, state]);

  const value = useMemo(
    () => ({
      ready,
      state,
      session,
      activeClientId,
      activeClient,
      inventory,
      shoppingList,
      challengeState,
      pendingCount,
      demoInventory,
      demoShoppingList,
      demoChallenges,
      adminSelectedClientId,
      setAdminSelectedClientId,
      login,
      register,
```

```
      requestPasswordReset,
      createClientByAdmin,
      setClientApproval,
      logout,
      addInventory,
      deleteInventory,
      addShopping,
      toggleShopping,
      deleteShopping,
      toggleChallenge,
      exportBackup,
      backendMode: firebaseMode ? 'firebase' : 'local',
    }),
    [
      ready,
      state,
      session,
      activeClientId,
      activeClient,
      inventory,
      shoppingList,
      challengeState,
      pendingCount,
      demoInventory,
      demoShoppingList,
      demoChallenges,
      adminSelectedClientId,
      login,
      register,
      requestPasswordReset,
      createClientByAdmin,
      setClientApproval,
      logout,
      addInventory,
      deleteInventory,
      addShopping,
      toggleShopping,
      deleteShopping,
      toggleChallenge,
      exportBackup,
      firebaseMode,
    ]
  );

  return <AppStoreContext.Provider value={value}>{children}</AppStoreContext.Provider>;
}

export function useAppStore() {
  const context = useContext(AppStoreContext);

  if (!context) {
    throw new Error('useAppStore deve ser usado dentro de AppStoreProvider.');
  }

  return context;
}
```

Arquivo: src/services/backendAdapter.js

```
const provider = (import.meta.env.VITE_BACKEND_PROVIDER || 'local').toLowerCase();

export const backendAdapter = {
  provider,

  isRemoteEnabled() {
    return provider !== 'local';
  },

  isFirebase() {
    return provider === 'firebase';
  },

  async sync() {
    if (provider === 'local') {
      return { synced: false, reason: 'provider-local' };
    }

    if (provider === 'firebase') {
      return { synced: true, reason: 'provider-firebase' };
    }

    return { synced: false, reason: 'provider-not-supported' };
  },
};
```

Arquivo: src/services/authService.js

```
import { createId } from '../utils/ids';

function isApprovedClient(account) {
  const status = account?.approvalStatus || 'approved';
  return status === 'approved';
}

function baseClientChallenges() {
  return {
    completed: [],
    current: [
      'Cadastrar os primeiros itens do estoque',
      'Montar a primeira lista de compras',
      'Revisar itens com validade mais proxima',
    ],
  };
}

export function loginWithMode(state, mode, email, password) {
  const source = mode === 'admin' ? state.adminAccounts : state.clientAccounts;
  const account = source.find((item) => item.email === email && item.password ===
password);

  if (!account) {
    return { ok: false, error: 'Nao encontramos esse acesso. Confira e-mail e senha.' };
```

```
  }

  if (mode === 'client' && !isApprovedClient(account)) {
    const status = account.approvalStatus || 'pending';

    if (status === 'pending') {
      return { ok: false, error: 'Seu cadastro esta pendente de aprovacao do
administrador.' };
    }

    if (status === 'rejected') {
      return { ok: false, error: 'Seu cadastro foi reprovado. Entre em contato com o
administrador.' };
    }

    return { ok: false, error: 'Sua conta de cliente ainda nao foi aprovada.' };
  }

  return { ok: true, account };
}

export function registerClientAccount(state, form) {
  const { name, email, password, businessType } = form;

  if (!name || !email || !password) {
    return { ok: false, error: 'Preencha nome, e-mail e senha para criar sua conta.' };
  }

  if (state.clientAccounts.some((account) => account.email === email)) {
    return { ok: false, error: 'Esse e-mail ja esta cadastrado.' };
  }

  const id = createId('cliente');
  const newClient = {
    id,
    role: 'client',
    name,
    email,
    password,
    businessType,
    approvalStatus: 'pending',
  };

  const nextState = {
    ...state,
    clientAccounts: [...state.clientAccounts, newClient],
    inventories: { ...state.inventories, [id]: [] },
    shoppingLists: { ...state.shoppingLists, [id]: [] },
    challenges: {
      ...state.challenges,
      [id]: baseClientChallenges(),
    },
  };

  return {
    ok: true,
    requiresApproval: true,
    message: 'Cadastro criado. Aguarde aprovacao do administrador para acessar o sistema.',
    nextState,
  };
}
```

```
export function createClientAccountByAdmin(state, form) {
  const { name, email, password, businessType } = form;

  if (!name || !email || !password) {
    return { ok: false, error: 'Preencha nome, e-mail e senha para cadastrar o cliente.' };
  }

  if (state.clientAccounts.some((account) => account.email === email)) {
    return { ok: false, error: 'Ja existe um cliente com esse e-mail.' };
  }

  if (state.adminAccounts.some((account) => account.email === email)) {
    return { ok: false, error: 'Esse e-mail ja esta em uso por um administrador.' };
  }

  const id = createId('cliente');
  const newClient = {
    id,
    role: 'client',
    name,
    email,
    password,
    businessType,
    approvalStatus: 'approved',
  };

  const nextState = {
    ...state,
    clientAccounts: [...state.clientAccounts, newClient],
    inventories: { ...state.inventories, [id]: [] },
    shoppingLists: { ...state.shoppingLists, [id]: [] },
    challenges: {
      ...state.challenges,
      [id]: baseClientChallenges(),
    },
  };

  return { ok: true, nextState, account: newClient };
}
```

Arquivo: src/services/firebaseClient.js

```
import { deleteApp, getApps, initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';

export const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
};

export const isFirebaseConfigured = Object.values(firebaseConfig).every(Boolean);

export const firebaseApp = isFirebaseConfigured ? initializeApp(firebaseConfig) : null;
export const auth = firebaseApp ? getAuth(firebaseApp) : null;
```

```
export const db = firebaseApp ? getFirestore(firebaseApp) : null;

export function assertFirebaseReady() {
  if (!isFirebaseConfigured || !auth || !db) {
    throw new Error('Firebase nao configurado. Preencha o arquivo .env.local com as chaves
VITE_FIREBASE_*.');
  }
}

export function createSecondaryAuthApp(name) {
  assertFirebaseReady();

  const appName = name || 'secondary-auth-${Date.now()}';
  const existing = getApps().find((app) => app.name === appName);
  const app = existing || initializeApp(firebaseConfig, appName);

  return {
    app,
    auth: getAuth(app),
    db: getFirestore(app),
  };
}

export async function disposeFirebaseApp(app) {
  if (!app || app.name === '[DEFAULT]') return;
  await deleteApp(app);
}
```

Arquivo: src/services/firebaseAuthService.js

```
import {
  createUserWithEmailAndPassword,
  deleteUser,
  onAuthStateChanged,
  reload,
  sendEmailVerification,
  sendPasswordResetEmail,
  signInWithEmailAndPassword,
  signOut,
} from 'firebase/auth';
import { doc, getDoc, serverTimestamp, setDoc } from 'firebase/firestore';
import { assertFirebaseReady, auth, createSecondaryAuthApp, db, disposeFirebaseApp } from
'./firebaseClient';

const APPROVAL_STATUS = {
  PENDING: 'pending',
  APPROVED: 'approved',
  REJECTED: 'rejected',
};

function mapAuthError(error) {
  const code = error?.code || '';

  if (code === 'auth/invalid-credential') {
    return 'E-mail ou senha invalidos.';
  }
  if (code === 'auth/invalid-email') {
    return 'E-mail invalido.';
  }
  if (code === 'auth/user-not-found') {
```

```
    return 'Conta nao encontrada.';
  }
  if (code === 'auth/wrong-password') {
    return 'Senha incorreta.';
  }
  if (code === 'auth/email-already-in-use') {
    return 'Esse e-mail ja esta em uso.';
  }
  if (code === 'auth/weak-password') {
    return 'A senha precisa ter pelo menos 6 caracteres.';
  }
  if (code === 'auth/operation-not-allowed') {
    return 'Login por e-mail/senha nao esta habilitado no Firebase Authentication.';
  }
  if (code === 'auth/unauthorized-domain') {
    return 'Dominio nao autorizado no Firebase Authentication. Adicione o dominio do site
em Authorized domains.';
  }
  if (code === 'auth/network-request-failed') {
    return 'Falha de rede ao falar com o Firebase. Verifique sua conexao e tente
novamente.';
  }
  if (code === 'auth/too-many-requests') {
    return 'Muitas tentativas em pouco tempo. Aguarde alguns minutos e tente novamente.';
  }
  if (code === 'permission-denied') {
    return 'Permissao negada no Firestore. Revise as regras do banco.';
  }
  if (code === 'failed-precondition') {
    return 'O Firestore nao esta configurado corretamente para essa operacao.';
  }
  if (code === 'auth/invalid-api-key') {
    return 'Chave da API invalida. Revise os Secrets/Variaveis do deploy.';
  }
  if (code === 'auth/invalid-continue-uri' || code === 'auth/unauthorized-continue-uri') {
    return 'Falha ao enviar verificacao de e-mail. Revise os dominios autorizados no
Firebase.';
  }

  return 'Nao foi possivel concluir a autenticacao agora. (${code || 'erro-
desconhecido'})';
}

function toSessionAccount(uid, profile, emailVerified = false) {
  return {
    id: uid,
    role: profile.role,
    name: profile.name || profile.email || 'Usuario',
    email: profile.email || '',
    businessType: profile.businessType || 'Operacao',
    approvalStatus: profile.approvalStatus || APPROVAL_STATUS.APPROVED,
    emailVerified,
  };
}

function approvalError(status) {
  if (status === APPROVAL_STATUS.PENDING) {
    return 'Seu cadastro esta pendente de aprovacao do administrador.';
  }

  if (status === APPROVAL_STATUS.REJECTED) {
```

```
    return 'Seu cadastro foi reprovado. Entre em contato com o administrador.';
  }

  return 'Sua conta de cliente ainda nao foi aprovada.';
}

async function getProfile(uid) {
  const snapshot = await getDoc(doc(db, 'users', uid));
  if (!snapshot.exists()) return null;
  return snapshot.data();
}

async function createClientProfile(firestoreDb, uid, form, approvalStatus, metadata = {}) {
  const { name, email, businessType } = form;

  const profile = {
    role: 'client',
    name,
    email,
    businessType,
    approvalStatus,
    createdAt: serverTimestamp(),
    ...metadata,
  };

  await setDoc(doc(firestoreDb, 'users', uid), profile, { merge: true });

  return profile;
}

export async function loginWithFirebase(mode, email, password) {
  if (!email || !password) {
    return { ok: false, error: 'Preencha e-mail e senha.' };
  }

  try {
    assertFirebaseReady();
    const credential = await signInWithEmailAndPassword(auth, email, password);
    const profile = await getProfile(credential.user.uid);

    if (!profile) {
      await signOut(auth);
      return { ok: false, error: 'Perfil nao encontrado no banco. Verifique o cadastro.' };
    }

    if (mode === 'admin' && profile.role !== 'admin') {
      await signOut(auth);
      return { ok: false, error: 'Esta conta nao possui acesso administrativo.' };
    }

    if (mode === 'client' && profile.role !== 'client') {
      await signOut(auth);
      return { ok: false, error: 'Use o acesso de administrador para esta conta.' };
    }

    if (mode === 'client') {
      await reload(credential.user);

      if (!credential.user.emailVerified) {
        await signOut(auth);
        return { ok: false, error: 'Confirme seu e-mail antes de entrar. Verifique sua
```

```
caixa de entrada.' };
      }

      await setDoc(
        doc(db, 'users', credential.user.uid),
        {
          emailVerified: true,
          emailVerifiedAt: serverTimestamp(),
          updatedAt: serverTimestamp(),
        },
        { merge: true }
      );

      const status = profile.approvalStatus || APPROVAL_STATUS.APPROVED;
      if (status !== APPROVAL_STATUS.APPROVED) {
        await signOut(auth);
        return { ok: false, error: approvalError(status) };
      }
    }

    return {
      ok: true,
      account: toSessionAccount(credential.user.uid, profile,
credential.user.emailVerified),
    };
  } catch (error) {
    console.error('loginWithFirebase error:', error);
    return { ok: false, error: mapAuthError(error) };
  }
}

export async function registerClientWithFirebase(form) {
  const { name, email, password, businessType } = form;

  if (!name || !email || !password) {
    return { ok: false, error: 'Preencha nome, e-mail e senha para criar sua conta.' };
  }

  let secondaryApp = null;
  let secondaryAuth = null;
  let secondaryDb = null;
  let createdUser = null;

  const defaultPendingMessage =
    'Cadastro enviado. Confirme seu e-mail e aguarde a autorizacao de um administrador.';

  try {
    assertFirebaseReady();

    const secondaryContext = createSecondaryAuthApp('public-register-${Date.now()}');
    secondaryApp = secondaryContext.app;
    secondaryAuth = secondaryContext.auth;
    secondaryDb = secondaryContext.db;

    const credential = await createUserWithEmailAndPassword(secondaryAuth, email,
password);
    createdUser = credential.user;

    await sendEmailVerification(createdUser);

    await createClientProfile(
```

```
      secondaryDb,
      createdUser.uid,
      { name, email, businessType },
      APPROVAL_STATUS.PENDING,
      {
        requestedAt: serverTimestamp(),
        emailVerified: false,
        verificationEmailSentAt: serverTimestamp(),
      }
    );

    return {
      ok: true,
      requiresApproval: true,
      message: defaultPendingMessage,
    };
  } catch (error) {
    if (error?.code === 'auth/email-already-in-use' && secondaryAuth && secondaryDb) {
      try {
        const recovery = await signInWithEmailAndPassword(secondaryAuth, email, password);
        const recoveryUser = recovery.user;

        if (!recoveryUser.emailVerified) {
          await sendEmailVerification(recoveryUser);
        }

        const profileSnapshot = await getDoc(doc(secondaryDb, 'users', recoveryUser.uid));

        if (!profileSnapshot.exists()) {
          await createClientProfile(
            secondaryDb,
            recoveryUser.uid,
            { name, email, businessType },
            APPROVAL_STATUS.PENDING,
            {
              requestedAt: serverTimestamp(),
              emailVerified: Boolean(recoveryUser.emailVerified),
              verificationEmailSentAt: serverTimestamp(),
            }
          );
        } else {
          const existingProfile = profileSnapshot.data() || {};

          if (existingProfile.role !== 'client') {
            return { ok: false, error: 'Esse e-mail pertence a um administrador.' };
          }

          if (existingProfile.approvalStatus === APPROVAL_STATUS.APPROVED) {
            return { ok: false, error: 'Esse e-mail ja possui conta ativa. Use a tela de
login.' };
          }

          await setDoc(
            doc(secondaryDb, 'users', recoveryUser.uid),
            {
              name,
              email,
              businessType,
              emailVerified: Boolean(recoveryUser.emailVerified),
              verificationEmailSentAt: serverTimestamp(),
              updatedAt: serverTimestamp(),
```

```
            },
            { merge: true }
          );
        }

        return {
          ok: true,
          requiresApproval: true,
          message:
            'Encontramos um cadastro anterior com este e-mail. Reenviamos a verificacao e
mantivemos seu cadastro em analise.',
        };
      } catch (recoveryError) {
        if (recoveryError?.code === 'auth/wrong-password' || recoveryError?.code ===
'auth/invalid-credential') {
          return {
            ok: false,
            error:
              'Esse e-mail ja esta em uso. Se a conta for sua, use a senha original ou
recupere o acesso antes de tentar novo cadastro.',
          };
        }
      }
    }

    if (createdUser) {
      try {
        await deleteUser(createdUser);
      } catch {
        // noop
      }
    }

    console.error('registerClientWithFirebase error:', error);
    return { ok: false, error: mapAuthError(error) };
  } finally {
    if (secondaryAuth) {
      signOut(secondaryAuth).catch(() => null);
    }

    if (secondaryApp) {
      disposeFirebaseApp(secondaryApp).catch(() => null);
    }
  }
}

export async function sendPasswordResetWithFirebase(email) {
  if (!email) {
    return { ok: false, error: 'Informe o e-mail para recuperar a senha.' };
  }

  try {
    assertFirebaseReady();
    await sendPasswordResetEmail(auth, email);

    return {
      ok: true,
      message: 'Se existir uma conta com este e-mail, enviamos o link de redefinicao de
senha.',
    };
  } catch (error) {
```

```
    if (error?.code === 'auth/user-not-found') {
      return {
        ok: true,
        message: 'Se existir uma conta com este e-mail, enviamos o link de redefinicao de
senha.',
      };
    }

    return { ok: false, error: mapAuthError(error) };
  }
}
export async function createClientByAdminWithFirebase(form, adminAccount) {
  const { name, email, password, businessType } = form;

  if (!adminAccount || adminAccount.role !== 'admin') {
    return { ok: false, error: 'Apenas administradores podem cadastrar clientes.' };
  }

  if (!name || !email || !password) {
    return { ok: false, error: 'Preencha nome, e-mail e senha para cadastrar o cliente.' };
  }

  let secondaryApp = null;
  let secondaryAuth = null;

  try {
    assertFirebaseReady();

    const secondaryContext = createSecondaryAuthApp();
    secondaryApp = secondaryContext.app;
    secondaryAuth = secondaryContext.auth;

    const credential = await createUserWithEmailAndPassword(secondaryAuth, email,
password);
    const uid = credential.user.uid;

    await createClientProfile(
      db,
      uid,
      { name, email, businessType },
      APPROVAL_STATUS.APPROVED,
      {
        approvedAt: serverTimestamp(),
        approvedBy: adminAccount.id,
        createdByAdminAt: serverTimestamp(),
        emailVerified: true,
      }
    );

    await signOut(secondaryAuth);
    await disposeFirebaseApp(secondaryApp);

    return {
      ok: true,
      account: toSessionAccount(
        uid,
        {
          role: 'client',
          name,
          email,
          businessType,
```

```
          approvalStatus: APPROVAL_STATUS.APPROVED,
        },
        true
      ),
    };
  } catch (error) {
    console.error('createClientByAdminWithFirebase error:', error);

    if (secondaryAuth) {
      try {
        await signOut(secondaryAuth);
      } catch {
        // noop
      }
    }

    if (secondaryApp) {
      try {
        await disposeFirebaseApp(secondaryApp);
      } catch {
        // noop
      }
    }

    return { ok: false, error: mapAuthError(error) };
  }
}

export function subscribeAuthSession(onChange) {
  assertFirebaseReady();

  return onAuthStateChanged(auth, async (user) => {
    if (!user) {
      onChange(null);
      return;
    }

    try {
      const profile = await getProfile(user.uid);
      if (!profile) {
        onChange(null);
        return;
      }

      if (profile.role === 'client') {
        await reload(user);

        if (!user.emailVerified) {
          await signOut(auth);
          onChange(null);
          return;
        }

        if (profile.approvalStatus && profile.approvalStatus !== APPROVAL_STATUS.APPROVED)
{
          await signOut(auth);
          onChange(null);
          return;
        }
      }
```

```
      onChange(toSessionAccount(user.uid, profile, user.emailVerified));
    } catch {
      onChange(null);
    }
  });
}

export async function logoutFirebase() {
  assertFirebaseReady();
  await signOut(auth);
}

export { APPROVAL_STATUS };
```

Arquivo: src/services/firebaseDataService.js

```
import {
  collection,
  deleteDoc,
  doc,
  getDoc,
  getDocs,
  onSnapshot,
  query,
  serverTimestamp,
  setDoc,
  updateDoc,
  where,
} from 'firebase/firestore';
import { createId } from '../utils/ids';
import { assertFirebaseReady, db } from './firebaseClient';

const defaultChallengeState = {
  completed: [],
  current: [
    'Cadastrar os primeiros itens do estoque',
    'Montar a primeira lista de compras',
    'Revisar itens com validade mais proxima',
  ],
};

function userDocRef(uid) {
  return doc(db, 'users', uid);
}

function clientDocRef(clientId) {
  return doc(db, 'clients', clientId);
}

function inventoryCollection(clientId) {
  return collection(db, 'clients', clientId, 'inventory');
}

function shoppingCollection(clientId) {
  return collection(db, 'clients', clientId, 'shopping');
}

function challengesDocRef(clientId) {
```

```
  return doc(db, 'clients', clientId, 'meta', 'challenges');
}

function normalizeChallenges(raw) {
  if (!raw) return { ...defaultChallengeState };

  return {
    completed: Array.isArray(raw.completed) ? raw.completed : [],
    current: Array.isArray(raw.current) && raw.current.length ? raw.current :
defaultChallengeState.current,
  };
}

function mapClientProfile(snapshot) {
  const data = snapshot.data() || {};
  return {
    id: snapshot.id,
    role: 'client',
    name: data.name || data.email || 'Cliente',
    email: data.email || '',
    businessType: data.businessType || 'Operacao',
    approvalStatus: data.approvalStatus || 'approved',
  };
}

function mapCollection(snapshot) {
  return snapshot.docs.map((item) => {
    const data = item.data() || {};
    return { id: item.id, ...data };
  });
}

export async function ensureClientData(clientId) {
  assertFirebaseReady();

  await setDoc(clientDocRef(clientId), { updatedAt: serverTimestamp() }, { merge: true });

  const challengesSnapshot = await getDoc(challengesDocRef(clientId));
  if (!challengesSnapshot.exists()) {
    await setDoc(challengesDocRef(clientId), defaultChallengeState, { merge: true });
  }
}

export async function loadClientData(clientId) {
  assertFirebaseReady();
  await ensureClientData(clientId);

  const [inventorySnapshot, shoppingSnapshot, challengesSnapshot] = await Promise.all([
    getDocs(inventoryCollection(clientId)),
    getDocs(shoppingCollection(clientId)),
    getDoc(challengesDocRef(clientId)),
  ]);

  return {
    inventory: mapCollection(inventorySnapshot),
    shoppingList: mapCollection(shoppingSnapshot),
    challenges: normalizeChallenges(challengesSnapshot.exists() ? challengesSnapshot.data()
: null),
  };
}
```

```
export function subscribeClientAccounts(onChange) {
  assertFirebaseReady();

  const clientsQuery = query(collection(db, 'users'), where('role', '==', 'client'));

  return onSnapshot(clientsQuery, (snapshot) => {
    const clients = snapshot.docs.map(mapClientProfile).sort((a, b) => {
      if (a.approvalStatus !== b.approvalStatus) {
        if (a.approvalStatus === 'pending') return -1;
        if (b.approvalStatus === 'pending') return 1;
      }

      return a.name.localeCompare(b.name);
    });

    onChange(clients);
  });
}

export function subscribeClientData(clientId, onChange) {
  assertFirebaseReady();

  const cache = {
    inventory: [],
    shoppingList: [],
    challenges: { ...defaultChallengeState },
  };

  ensureClientData(clientId).catch(() => null);

  const unsubscribeInventory = onSnapshot(inventoryCollection(clientId), (snapshot) => {
    cache.inventory = mapCollection(snapshot);
    onChange({ ...cache });
  });

  const unsubscribeShopping = onSnapshot(shoppingCollection(clientId), (snapshot) => {
    cache.shoppingList = mapCollection(snapshot);
    onChange({ ...cache });
  });

  const unsubscribeChallenges = onSnapshot(challengesDocRef(clientId), (snapshot) => {
    cache.challenges = normalizeChallenges(snapshot.exists() ? snapshot.data() : null);
    onChange({ ...cache });
  });

  return () => {
    unsubscribeInventory();
    unsubscribeShopping();
    unsubscribeChallenges();
  };
}

export async function updateClientApprovalStatus(clientId, approvalStatus, adminId) {
  assertFirebaseReady();

  await setDoc(
    userDocRef(clientId),
    {
      approvalStatus,
      approvedBy: adminId || null,
      approvedAt: serverTimestamp(),
```

```
      updatedAt: serverTimestamp(),
    },
    { merge: true }
  );
}

export async function addInventoryItem(clientId, item) {
  assertFirebaseReady();

  const payload = {
    ...item,
    id: item.id || createId('item'),
    createdAt: serverTimestamp(),
  };

  await setDoc(doc(inventoryCollection(clientId), payload.id), payload, { merge: true });
  return payload;
}

export async function deleteInventoryItem(clientId, itemId) {
  assertFirebaseReady();
  await deleteDoc(doc(inventoryCollection(clientId), itemId));
}

export async function addShoppingItem(clientId, item) {
  assertFirebaseReady();

  const payload = {
    ...item,
    id: item.id || createId('shop'),
    checked: Boolean(item.checked),
    createdAt: serverTimestamp(),
  };

  await setDoc(doc(shoppingCollection(clientId), payload.id), payload, { merge: true });
  return payload;
}

export async function toggleShoppingItem(clientId, itemId) {
  assertFirebaseReady();

  const itemRef = doc(shoppingCollection(clientId), itemId);
  const snapshot = await getDoc(itemRef);
  if (!snapshot.exists()) return;

  const current = snapshot.data() || {};
  await updateDoc(itemRef, { checked: !Boolean(current.checked), updatedAt:
serverTimestamp() });
}

export async function deleteShoppingItem(clientId, itemId) {
  assertFirebaseReady();
  await deleteDoc(doc(shoppingCollection(clientId), itemId));
}

export async function toggleChallengeItem(clientId, challenge) {
  assertFirebaseReady();

  await ensureClientData(clientId);
  const challengeRef = challengesDocRef(clientId);
  const snapshot = await getDoc(challengeRef);
```

```
  const current = normalizeChallenges(snapshot.exists() ? snapshot.data() : null);

  const exists = current.completed.includes(challenge);
  const completed = exists
    ? current.completed.filter((item) => item !== challenge)
    : [...current.completed, challenge];

  const next = {
    ...current,
    completed,
    updatedAt: serverTimestamp(),
  };

  await setDoc(challengeRef, next, { merge: true });

  return {
    completed,
    current: current.current,
  };
}

export async function getUserProfile(uid) {
  assertFirebaseReady();
  const snapshot = await getDoc(userDocRef(uid));
  if (!snapshot.exists()) return null;
  return snapshot.data();
}
```

Arquivo: src/services/kitchenService.js

```
export function recipeSuggestions(items) {
  const names = items.map((item) => item.name.toLowerCase());
  const hasAny = (...terms) => terms.some((term) => names.some((name) =>
name.includes(term)));

  const suggestions = [];

  if (hasAny('tomate', 'alface', 'queijo')) {
    suggestions.push({
      title: 'Combo de saladas ou prato leve',
      description: 'Aproveite tomate, alface e queijo para gerar saida rapida dos itens
frescos.',
      priority: 'Alta',
    });
  }

  if (hasAny('pao')) {
    suggestions.push({
      title: 'Combo promocional de padaria',
      description: 'Use os paes do dia em combos ou acompanhamentos para evitar perda
imediata.',
      priority: 'Alta',
    });
  }

  if (hasAny('frango', 'arroz', 'queijo')) {
    suggestions.push({
      title: 'Escondidinho ou arroz cremoso',
      description: 'Reaproveitamento util para itens que ja estao na operacao.',
```

```
      priority: 'Media',
    });
  }

  if (hasAny('leite', 'chocolate')) {
    suggestions.push({
      title: 'Bebidas e sobremesas especiais',
      description: 'Use leite e chocolate para aumentar venda de bebidas sazonais e
sobremesas.',
      priority: 'Media',
    });
  }

  if (!suggestions.length) {
    suggestions.push(
      {
        title: 'Montar prato do dia com itens criticos',
        description: 'Priorize ingredientes com menor prazo de validade e gere uma oferta
especifica.',
        priority: 'Alta',
      },
      {
        title: 'Criar promocao relampago',
        description: 'Use itens com estoque alto em combos ou kits para aumentar giro.',
        priority: 'Media',
      }
    );
  }

  return suggestions;
}

export function challengeTips() {
  return [
    'Organize sua despensa por prioridade de validade, nao apenas por categoria.',
    'Crie um prato do dia usando pelo menos 2 itens que vencem em ate 48 horas.',
    'Revise a lista de compras antes de cada reposicao para evitar compras duplicadas.',
    'Defina uma meta semanal de reducao de perdas e acompanhe o resultado no painel.',
  ];
}
```

Arquivo: src/services/storageService.js

```
export function loadState(storageKey, defaultState) {
  try {
    const raw = localStorage.getItem(storageKey);
    if (!raw) return defaultState;

    const parsed = JSON.parse(raw);
    return {
      ...defaultState,
      ...parsed,
      inventories: { ...defaultState.inventories, ...(parsed.inventories || {}) },
      shoppingLists: { ...defaultState.shoppingLists, ...(parsed.shoppingLists || {}) },
      challenges: { ...defaultState.challenges, ...(parsed.challenges || {}) },
    };
  } catch {
    return defaultState;
  }
}
```

```
export function persistState(storageKey, state) {
  localStorage.setItem(storageKey, JSON.stringify(state));
}
```

Arquivo: src/utils/date.js

```
export function addDays(days) {
  const date = new Date();
  date.setDate(date.getDate() + days);
  return date.toISOString().slice(0, 10);
}

export function daysUntil(dateStr) {
  const today = new Date();
  today.setHours(0, 0, 0, 0);

  const target = new Date(dateStr);
  target.setHours(0, 0, 0, 0);

  return Math.round((target - today) / (1000 * 60 * 60 * 24));
}

export function statusFromExpiry(dateStr) {
  const diff = daysUntil(dateStr);

  if (diff < 0) {
    return { label: 'Vencido', tone: 'text-red-300 bg-red-500/15 border-red-500/20' };
  }

  if (diff === 0) {
    return { label: 'Vence hoje', tone: 'text-red-200 bg-red-500/15 border-red-500/20' };
  }

  if (diff <= 2) {
    return {
      label: `Vence em ${diff} dia${diff > 1 ? 's' : ''}`,
      tone: 'text-amber-200 bg-amber-500/15 border-amber-500/20',
    };
  }

  if (diff <= 5) {
    return { label: `Atencao em ${diff} dias`, tone: 'text-yellow-200 bg-yellow-500/15
border-yellow-500/20' };
  }

  return { label: 'Estavel', tone: 'text-emerald-200 bg-emerald-500/15 border-
emerald-500/20' };
}
```

Arquivo: src/utils/export.js

```
export function downloadJson(filename, data) {
  const blob = new Blob([JSON.stringify(data, null, 2)], { type:
'application/json;charset=utf-8' });
  const url = URL.createObjectURL(blob);

  const anchor = document.createElement('a');
  anchor.href = url;
  anchor.download = filename;
  document.body.appendChild(anchor);
```

```
  anchor.click();
  document.body.removeChild(anchor);

  URL.revokeObjectURL(url);
}
```

Arquivo: src/utils/ids.js

```
export function createId(prefix = 'id') {
  const randomPart = Math.random().toString(36).slice(2, 8);
  return `${prefix}-${Date.now()}-${randomPart}`;
}
```